Student ID: 24173338

Name: Sony Thapa Magar

GitHub Repository: https://github.com/sonythapaa/Anamoly-Detection-Using-Autoencoder

Autoencoders in Deep Learning for Anomaly Detection

What You Will Learn

In this tutorial, you will learn:

- Why supervised learning struggles with rare anomalies
- How autoencoders detect anomalies using reconstruction error and thresholds
- Visualizing original vs reconstructed patterns with autoencoders

Accessibility Note: All the figures and plots in this tutorial use a color palette that is colorblind-friendly, ensuring that the visualizations are clear and interpretable for everyone. In addition, we also use markers, line styles, and text labels, so you don't have to rely on color alone to understand the visuals.

1. Why Supervised Learning Fails for Rare and Unseen Anomalies

Imagine managing quality control at a factory. Thousands of parts are produced every hour, and only a few are defective. Your job is to find them. You could build a classifier: labelled examples of good parts, labelled examples of bad parts, train it to tell them apart. But defects are rare by definition. You might have fifty examples of each defect type. Next month there will be a new defect you have never seen. You cannot label what you have not encountered.

The fundamental constraint: in anomaly detection, you almost never have enough labelled anomaly examples to train a supervised classifier. What you always have plenty of is normal data. The question becomes: can you teach a system what normal looks like so thoroughly that anything abnormal stands out on its own?

This is the problem autoencoders solve. They learn a compact, efficient representation of normal data entirely from unlabelled examples. When presented with abnormal data, autoencoders produce a high reconstruction error, failing to reconstruct it accurately. The table below contrasts both approaches:

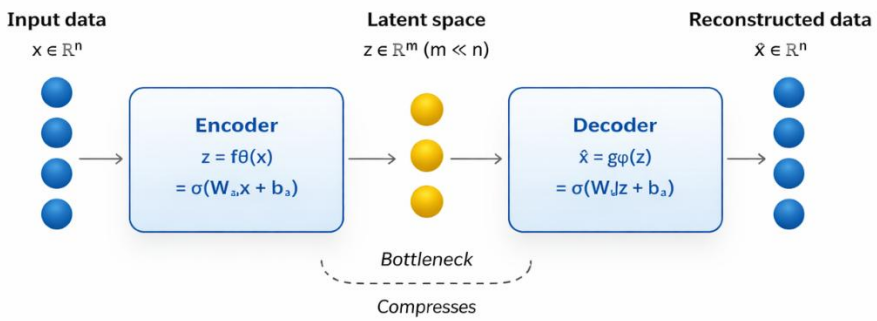
Supervised Classifier	Autoencoder (Unsupervised)
Needs labelled examples of every anomaly type	Trains on normal data only — zero anomaly labels needed
Cannot detect anomaly types not seen in training	Detects any anomaly type, including ones never seen before
Output is a class label	Output is reconstruction error — a single interpretable number

2. What is an Autoencoder?

An autoencoder is a neural network trained to reproduce its own input at the output layer. It is not predicting a label or a class — it is reconstructing what it was given. The critical constraint that makes this useful is the bottleneck.

2.1 Architecture

The network consists of two components: an Encoder, which compresses the input into a lower-dimensional latent representation (the bottleneck), and a Decoder, which reconstructs the original input from that compressed form.



<u>Encoder: compression</u>

The encoder is a neural network function f that takes a high-dimensional input x and maps it to a low-dimensional latent vector z. Its main job is to capture the most important features of the input while reducing its dimensionality. Let $x \in \mathbb{R}^n$ be an input vector. The encoder function f_θ maps x to a latent vector z:

$$z = f_\theta(x) = \sigma(W_e x + b_e)$$

where We is the weight matrix, be is the bias vector, σ is a non-linear activation function (such as ReLU, sigmoid, or tanh), and θ = {We, be} represents all learnable encoder parameters. Given input $x \in \mathbb{R}^n$, the encoder produces $z \in \mathbb{R}^m$ where $m \ll n$.

Bottleneck: the latent space

The latent vector z is the bottleneck — the single narrow bridge between encoder and decoder. The latent space represents the most important features of the input data in a reduced dimension. By forcing the data through this narrow bottleneck, the network is trained to learn efficient representations rather than simply copying the input.

Why the bottleneck matters for anomaly detection: because the encoder only ever sees normal training data, the latent space only contains representations of normal patterns. An anomalous input has no matching region in this space — the encoder maps it to the nearest normal pattern it knows, and the decoder reconstructs that normal pattern instead of the anomaly. The gap between anomalous input and normal reconstruction is the detection signal.

Decoder: reconstruction

The decoder gφ reconstructs the original input from the latent representation z:

$$\hat{x} = g_\varphi(z) = \sigma(W_d z + b_d)$$

where W_d is the decoder weight matrix, bd is the decoder bias, and $\varphi = \{W_d, b_d\}$ are the learnable decoder parameters. Note that θ and φ are entirely separate parameter sets — the encoder and decoder each learn independently during training.

What Encoders and Decoders Are Made Of

The encoder and decoder are both neural networks, but what they are built from depends on the data. For tabular data, Dense layers are used since each feature is independent. For image data, Convolutional layers (Conv2D) are used instead — pixels are spatially related and Conv2D filters detect local patterns like edges and textures that Dense layers would miss entirely. For sequential data, LSTM layers are used because order matters and LSTM cells remember what happened in previous time steps. In all cases the encoder compresses using these layers and the decoder does the exact reverse to reconstruct the input.

```python
input  = X_train.shape[1]   # 140
latent  = 16                 # bottleneck size

def build_autoencoder(input_dim: int, latent_dim: int):
    #  Encoder
    inputs = keras.Input(shape=(input_dim,), name="encoder_input")
    x = layers.Dense(64, activation="relu", name="enc_dense1")(inputs)
    x = layers.Dense(32, activation="relu", name="enc_dense2")(x)
    encoded = layers.Dense(latent_dim, activation="relu", name="bottleneck")(x)

    #  Decoder
    x = layers.Dense(32, activation="relu", name="dec_dense1")(encoded)
    x = layers.Dense(64, activation="relu", name="dec_dense2")(x)
    decoded = layers.Dense(input_dim, activation="sigmoid", name="decoder_output")(x)

    model = keras.Model(inputs, decoded, name="ECG_Autoencoder")
    return model
```

2.2 Reconstruction Loss

The model is trained to minimise the Mean Squared Error (MSE) between the original input x and the reconstruction $\hat{x}$:

$$L(x, \hat{x}) = \|x - \hat{x}\|^2 = \frac{1}{n} \sum_{i=1}^{n} (x - \hat{x})^2$$

MSE is appropriate here because the output is a continuous reconstruction, not a categorical prediction. This contrasts directly with cross-entropy loss used in classification, where outputs represent class probabilities. The loss function produces a scalar for each sample, measuring how well the model reconstructed its input.

2.3 Training Objective

The full training objective finds the parameter sets θ* and φ* that jointly minimise reconstruction error across all normal training examples:

$$\theta^*, \varphi^* = \text{argmin} \; E_x \left[L(x, \; g_\varphi(f_\theta(x))) \right]$$

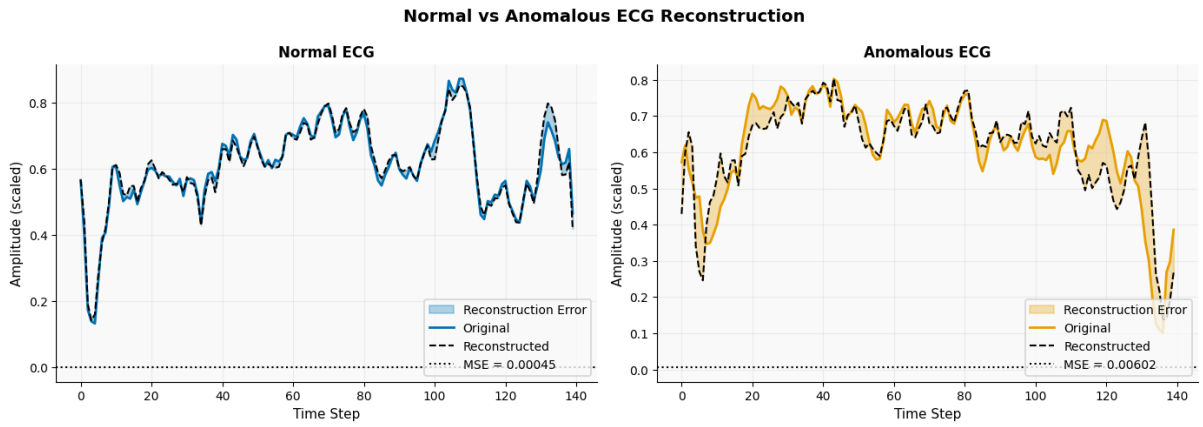
where E_x means the average loss calculated across all normal training samples, and g_φ(f_θ(x)) means passing x through the encoder then the decoder to get the reconstruction $\hat{x}$. The minimisation of this reconstruction error is done iteratively using gradient descent — common optimiser choices include SGD, RMSprop and Adam — all of which update θ and φ by backpropagating the gradient of L. The model is trained exclusively on normal data, X_normal = {x : label(x) = 1}, meaning it never sees an anomaly during training. This is intentional and is the key design decision that makes the whole approach work.

```python
# Split data into normal and anomalous
X_normal   = X_scaled[y == 1]
X_anomaly  = X_scaled[y == 0]

"""
  Train only on normal data!
  The autoencoder should learn the distribution of normal ECGs.
  It should never see anomalies during training.
"""
X_train, X_val = train_test_split(X_normal, test_size=0.1, random_state=SEED)
```

```python
autoencoder = build_autoencoder(input,latent)
autoencoder.compile(optimizer="adam", loss="mse")
autoencoder.summary()


#  Train the Autoencoder

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True
)

history = autoencoder.fit(
    X_train, X_train,              # Input == Target!
    epochs=100,
    batch_size=64,
    validation_data=(X_val, X_val),
    callbacks=[early_stop],
    verbose=1,
)
```

2.4 Real-World Applications

- **Manufacturing** — detecting scratches, cracks and misalignments on production lines
- **Healthcare** — flagging unusual patterns in MRI, ECGs or CT scans without requiring labeled disease data in advance.
- **Finance** — spotting suspicious transactions.
- **Cybersecurity** — identifying abnormal network activity
- **Computer vision research** — creating compact image representations that can be used for later tasks like recognition or finding similar images.



For the normal ECG, the autoencoder learns to reproduce the input closely, resulting in low reconstruction errors. For the anomalous ECG (used here for training demonstration), the reconstruction deviates more from the original, producing higher reconstruction errors. These errors indicate how well the model has learned to capture the patterns in the training data, but at this stage, no explicit anomaly detection or thresholding is being applied—the figure simply demonstrates the model's reconstruction performance.

3. Anomaly Detection Workflow

At this point the model is fully trained. All the maths from the previous section is now stored inside it. To detect anomalies, all we do is pass a new sample through the model, measure how well it reconstructs it, and compare that error to our threshold.

3.1 Anomaly Score

For each sample x, compute the reconstruction error as an anomaly score A(x):

$$\mathbf{A(x) = L(x, \hat{x}) = \|x - \hat{x}\|^2 = \frac{1}{n} \sum_{i=1}^{n} (x - \hat{x})^2}$$

A(x) is a scalar: a single number per sample. A high A(x) indicates poor reconstruction, which indicates the sample is likely anomalous. A low A(x) indicates the model reconstructed the sample well, meaning it lies within the learned normal distribution.

3.2 Decision Rule

A threshold τ converts the continuous anomaly score into a binary classification decision, directly mirroring the decision boundary concept from logistic regression:

```
ŷ  =  1  (NORMAL)    if  A(x) ≤ τ
ŷ  =  0  (ANOMALY)   if  A(x) > τ
```

Defining threshold:

Once the model is trained, we pass all the normal training samples through it and collect their reconstruction errors. From these errors we calculate the mean μ and standard deviation σ. The threshold is then set statistically from the training error distribution as:

```
τ  =  μ + kσ
```

where μ represents the mean of the reconstruction errors computed on the training set, σ represents the corresponding standard deviation, and k is a predefined constant set to 3.

Setting k = 3 comes from a well known rule in statistics — within 3 standard deviations of the mean sits 99.7% of normally distributed data. So anything beyond that boundary is considered too far from normal to be a genuine normal sample, and gets flagged as an anomaly.

This Gaussian approach is easy to use, but it assumes the reconstruction errors follow a symmetric bell curve. In real-world data, errors are often skewed or have multiple peaks, so this assumption can lead to mistakes. A more robust option is the percentile-based threshold. For example, using the 95th percentile sets a cutoff without assuming any specific distribution, making it less sensitive to outliers or unusual data.

Suppose, training reconstruction errors are:

$$\boldsymbol{E}_{\text{train}} = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \dots, \boldsymbol{e}_n\}$$

The p-th percentile threshold (e.g., 95th percentile) is defined as:

$$\boldsymbol{\tau}_p = \textbf{percentile}(\boldsymbol{E}_{\text{train}}, \boldsymbol{p})$$

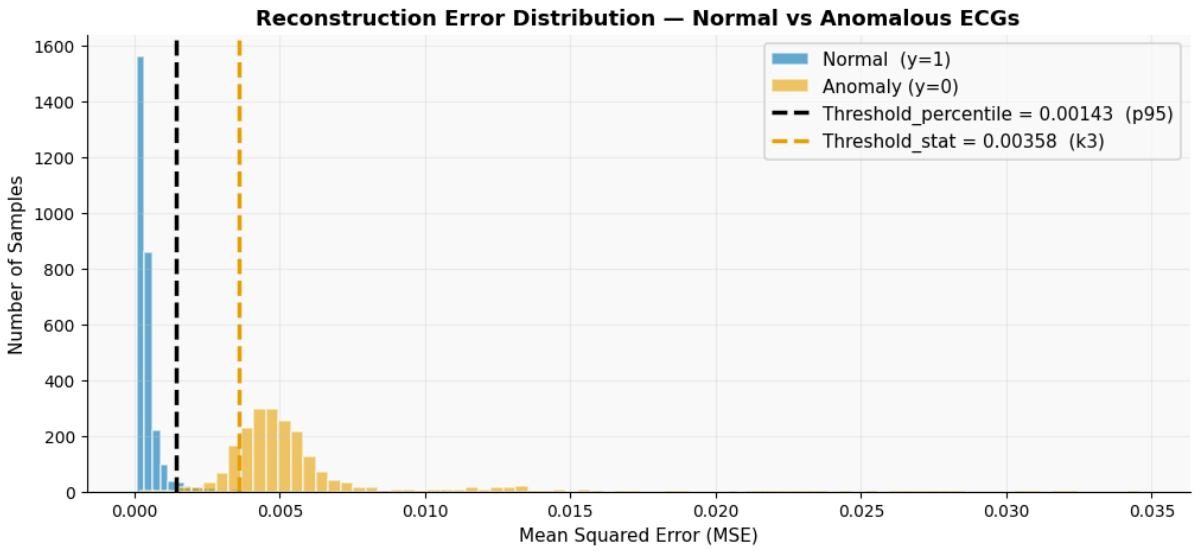
or more explicitly, if we sort the errors in ascending order:

$$\boldsymbol{E}_{\text{sorted}} = \{\boldsymbol{e}_{(1)}, \boldsymbol{e}_{(2)}, \dots, \boldsymbol{e}_{(n)}\}, \boldsymbol{e}_{(1)} \leq \boldsymbol{e}_{(2)} \leq \cdots \leq \boldsymbol{e}_{(n)}$$

then the p-th percentile is:

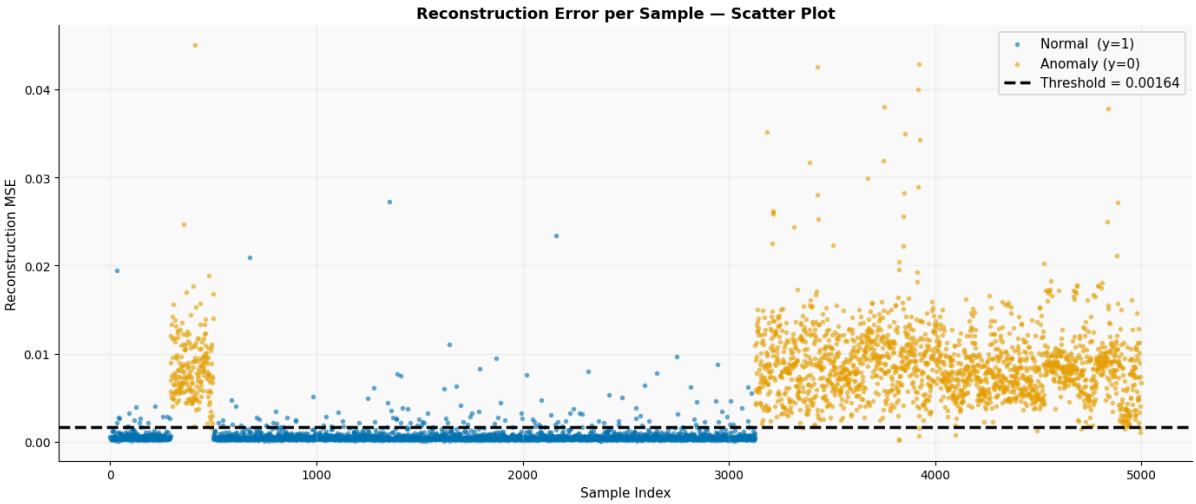
$$\boldsymbol{\tau}_p = \boldsymbol{e}_{\left\lceil \boldsymbol{n} \cdot \frac{\boldsymbol{p}}{\mathbf{100}} \right\rceil}$$

In our example, normal data errors aren't normally distributed and there are some extreme values. Because traditional statistical methods assume a normal distribution, these outliers push the average and spread higher, making the threshold too high. Using the 95th percentile instead gives a lower, more realistic cutoff that reflects most of the normal cases, making it easier to spot anomalies as shown on the figure.



The figure below shows how an autoencoder detects anomalies in ECG signals. Each point represents a sample, colored blue for normal and orange for abnormal. The dashed line marks the threshold: samples below it are considered normal, while those above are flagged as anomalous. Most normal ECGs have low reconstruction errors, whereas abnormal ECGs exceed the threshold, allowing the model to successfully identify unusual heart signals.

For more complex environments, this logic can be extended to dynamic thresholds that adapt to shifting data patterns, or to Extreme Value Theory (EVT) for modeling rare, high-impact events. Ultimately, the choice of methodology is a strategic one, balancing the need to suppress false alarms against the imperative to capture critical anomalies. This need for adaptive thresholds and advanced statistical modeling highlights why different autoencoder variants—such as Vanilla AE, Convolutional AE, LSTM AE, VAE, and Denoising AE—have been developed, each tailored to specific data types and environments, yet still subject to limitations like threshold sensitivity, overfitting, and reduced interpretability.



7. Strengths and Limitations

Strengths	Limitations
No anomaly labels required — trains on normal data only	Threshold τ selection is critical; wrong choice hurts performance
Detects novel anomaly types never seen before	Large bottleneck risks memorisation rather than learning
A(x) is a single interpretable anomaly score	Cannot explain why a sample is anomalous, only that it is
Scales to high-dimensional data: images, signals, logs	Assumes training data is clean (normal only)
Flexible architecture — Conv or LSTM layers can be added	May degrade on non-stationary or drifting data distributions

References

Chollet, F. (2021) *Deep Learning with Python*. 2nd edn. Shelter Island: Manning Publications.

Hinton, G.E. and Salakhutdinov, R.R. (2006) 'Reducing the dimensionality of data with neural networks', *Science*, 313(5786), pp. 504–507.

Pang, G., Chen, C., Liu, J. et al. (2021) 'Deep learning for anomaly detection: a review', *ACM Computing Surveys*, 54(2), pp. 1–38.

Gong, D., Liu, L., Le, V., Shao, C. and Ferreira, R.C. (2019) 'Memorizing normality to detect anomaly: memory-augmented deep autoencoder for unsupervised anomaly detection', in *Proceedings of the 36th International Conference on Machine Learning (ICML 2019)*.

Sakurada, M. and Yairi, T. (2014) 'Anomaly detection using autoencoders with nonlinear dimensionality reduction', in *Proceedings of the Machine Learning for Signal Processing Workshop (MLSP 2014)*, pp. 4–11.

Hawkins, S., He, H., Williams, G. and Baxter, R. (2002) 'Outlier detection using replicator neural networks', *Data Mining and Knowledge Discovery*, 6(1), pp. 63–82.

Goldstein, M. and Uchida, S. (2016) 'A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data', *PLoS ONE*, 11(4), e0152173.

Givnan, S., Chalmers, C., Fergus, P., Ortega-Martorell, S. and Whalley, T. (2022) 'Anomaly detection using autoencoder reconstruction upon industrial motors', *Sensors*, 22(9), p. 3166.